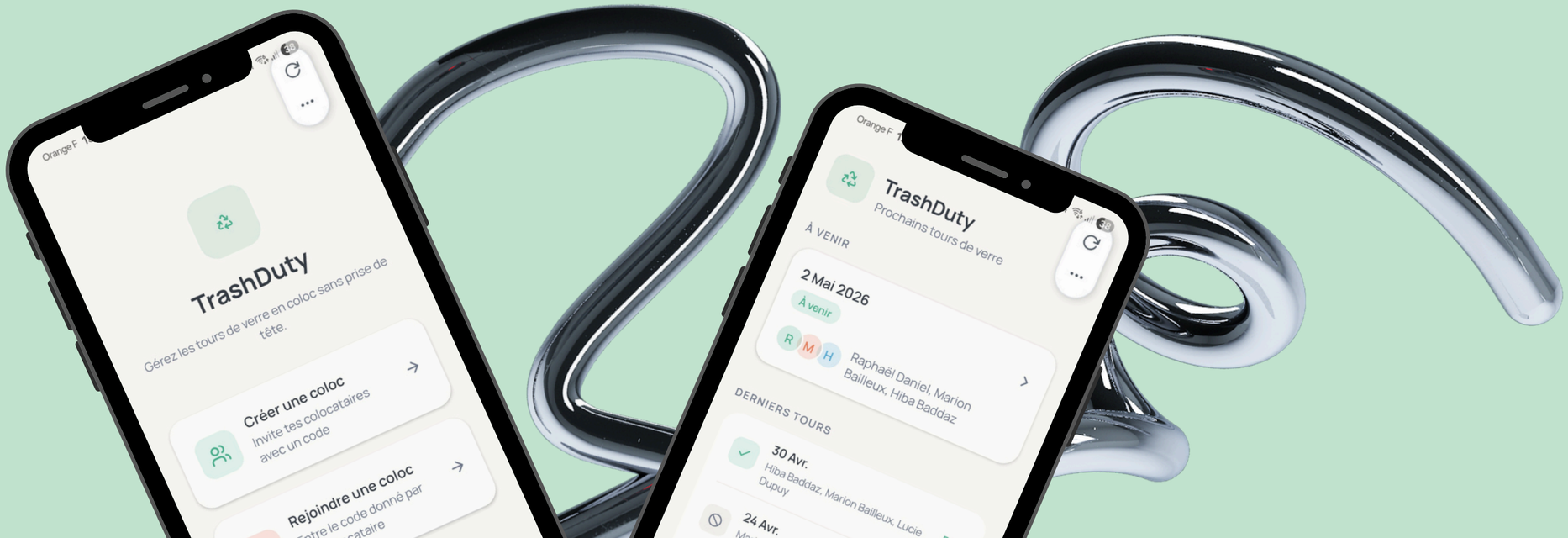
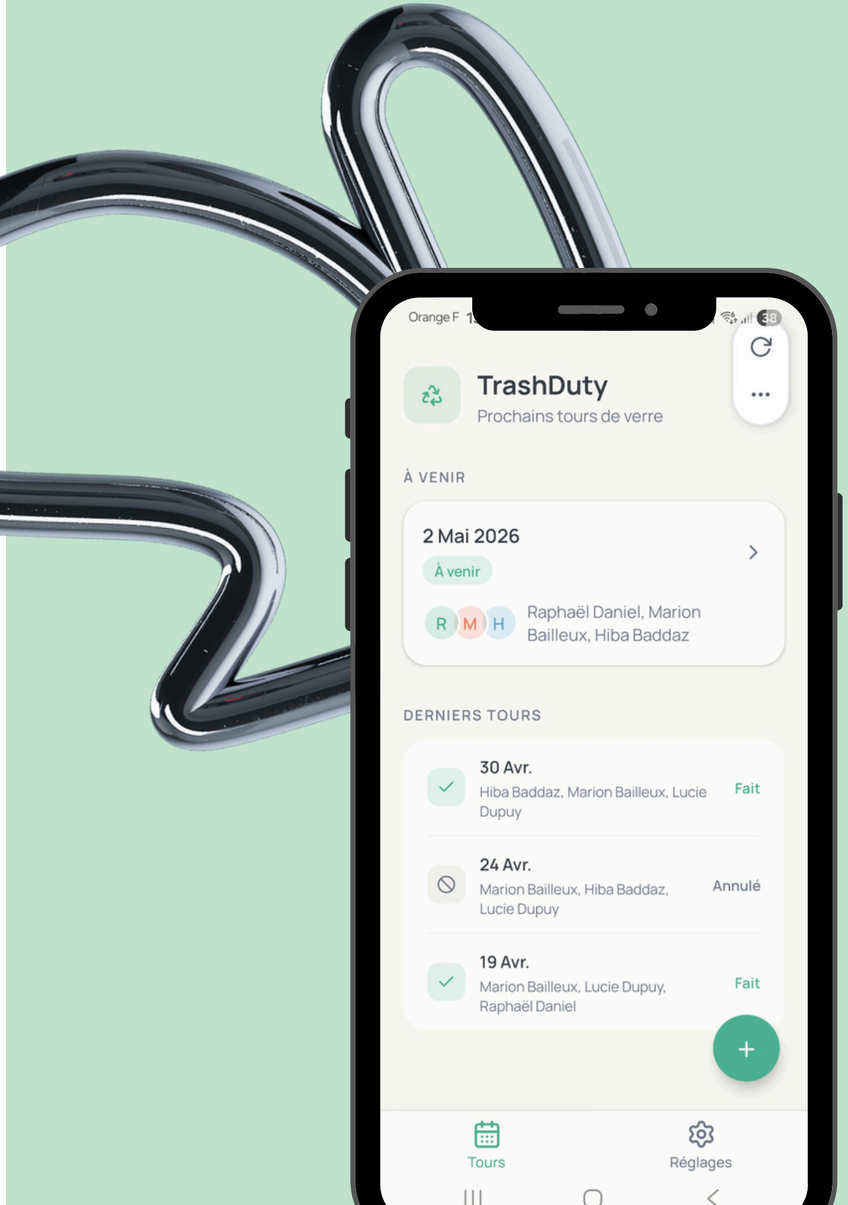
Présentation de l’application mobile

TrashDuty

Avril 2026





TrashDuty

L'app qui décide pour vous qui dépose le verre cette semaine.

Gère automatiquement le tour de dépôt du verre en colocation : attribution, rappels, historique.

Concept

Personnes en colocation

- Qui devait le faire cette semaine ?
- C'est toujours les mêmes qui le font.
- On a oublié de prévenir que quelqu'un était absent.

- Attribution automatique équitable basée sur l'historique
- Gestion des indisponibilités
- Rappels push automatiques
- Tout le monde voit la même info en temps réel

Architecture

UI (app/)

Expo Router, écrans, composants

État, loading, error

ViewModels (src/viewmodels/)

Appelle les UseCases

Logique métier pure

UseCases (src/usecases/)

Calculs, règles, validations

Firebase + cache local

Repositories + DataSources
(src/repositories/ + datasources/)

couche UI

ViewModels

addTourViewModel.ts

Gère l'état de l'écran de création d'un tour : chargement des membres de la coloc, sélection de la date, choix manuel ou automatique des responsables, sauvegarde du tour.

tourListViewModel.ts

Gère l'état de la liste des tours : souscription temps réel à Firebase, séparation en "à venir" et "passés", cache local pour le mode hors ligne, chargement des noms des membres.

tourDetailViewModel.ts

Gère l'état de l'écran de détail d'un tour : chargement du tour et du journal des modifications, actions (marquer fait, annuler, ajouter/retirer/régénérer un responsable).

settingsViewModel.ts

Gère l'état de l'écran des paramètres : chargement de la coloc et de ses membres, gestion des disponibilités, copie du code coloc, quitter la coloc, déconnexion.

couche logique métier

UseCases

authUseCase.ts

Règles d'inscription, connexion et déconnexion : crée l'utilisateur Firebase Auth et son profil Firestore à l'inscription.

colocUseCase.ts

Règles de création et de rejoindre une coloc : génère un code unique à 5 caractères, vérifie qu'un code existe avant de rejoindre.

tourAssignmentUseCase.ts

Algorithme d'attribution automatique des responsables : filtre les membres disponibles pour la date du tour, trie par ancienneté du dernier tour, sélectionne les N premiers.

tourDetailUseCase.ts

Règles des actions sur un tour : marquer comme fait, annuler, ajouter/retirer/régénérer un responsable. Chaque action écrit une entrée dans le journal et gère les notifications associées.

settingsUseCase.ts

Règles de gestion des membres : quitter une coloc ou retirer un membre retire l'utilisateur de tous les tours à venir où il était désigné, et supprime la coloc si elle se retrouve vide.

tourStatusUseCase.ts

Calcule l'affichage du badge de statut d'un tour (label et couleur) selon son statut (`pending`, `done`, `cancelled`) et sa date.

overdueReminderUseCase.ts

Gère les rappels quotidiens pour les tours en retard : programme des notifications pour les tours dont la date est passée mais pas encore marqués comme faits, annule ceux qui ne sont plus en retard.

couche données

DataSources

authRemoteDataSource.ts

Tous les appels Firebase Auth (inscription, connexion, déconnexion, écoute de l'état auth) et les lectures/écritures du profil utilisateur dans Firestore.

colocRemoteDataSource.ts

Lectures et écritures des documents de colocation dans Firestore : créer, lire par ID ou par code, ajouter/retirer un membre, supprimer.

tourRemoteDataSource.ts

Toutes les opérations Firestore sur les tours : créer, lire, souscriptions temps réel, mettre à jour le statut ou les responsables, gérer le journal des modifications et les IDs de notifications.

LOCAL

tourLocalDataSource.ts

Sauvegarde et lecture du cache des tours sur l'appareil. Utilisé pour afficher les données en mode hors ligne.

overdueReminderLocalDataSource.ts

Sauvegarde et lecture des IDs de notifications de rappel (par utilisateur et par tour) sur l'appareil. Permet d'annuler les bonnes notifications quand un tour est marqué fait.

Fonctionnalités

Authentification

Email / mot de passe via Firebase Authentication

- Firestore pour toutes les données
- Chaque opération async expose `isLoading`, `error`, `data`
- Subscriptions temps réel sur la liste des tours
- Cache local (AsyncStorage) pour le mode hors ligne

- Notifications push locales via `expo-notifications`
- Notification J-2 à 13h et Jour J à 13h pour chaque tour
- Annulation automatique si le tour est annulé
- Gérée silencieusement en cas de refus de permission

Choix techniques

Technologie	Choix	Raison
Framework	React Native (Expo)	Cross-platform iOS/Android, hot reload, accès natifs via modules Expo
Navigation	Expo Router	Basée sur les fichiers, gestion des groupes de routes (auth/onboarding/app)
Backend	Firebase (Auth + Firestore)	Temps réel, pas de serveur à gérer
Cache local	AsyncStorage	Persistance simple, mode hors ligne
Notifications	expo-notifications	Inclus dans expo
Gestion d'état	useState + hooks custom (ViewModels)	Simple, lisible, pas de lib externe

Problèmes rencontrés

Problème	Solution
Décalage timezone sur les dates d'indisponibilité	Comparaison en strings YYYY-MM-DD extraites en heure locale, sans passer par new Date()
Règles Firestore bloquaient des opérations légitimes (rejoindre une coloc, retirer un membre)	Ajustement progressif des règles : read ouvert à tous les connectés, write restreint selon le contexte
Problèmes de styles	Fignoler, chercher dans la doc, etc.

Améliorations

Fonctionnelles	Techniques
Notifications push distantes pour notifier les autres membres d'un tour	Tests unitaires sur les UseCases
Support d'autres types de corvées (poubelles, courses, ménage)	SyncManager : file d'attente des actions hors ligne à synchroniser au retour de la connexion
Statistiques par membre (nombre de tours effectués, taux de participation)	